

使用 Gemini CLI、BrowserMCP 和 Playwright 自動執行 UI 測試

來源：<https://codelabs.developers.google.com/agent-ai-testing?hl=zh-tw>

步驟數：11

本程式碼研究室會說明如何結合代理程式用戶端 (例如 Gemini CLI 和 Antigravity)、MCP 伺服器 and 代理程式技能，自動執行 UI 測試。

目錄

1. [簡介](#)
2. [必要條件](#)
3. [我們的示範應用程式](#)
4. [UI 測試的挑戰](#)
5. [MCP 救援](#)
6. [使用 BrowserMCP 自動化動作](#)
7. [使用 Skills 和 Playwright 自動化](#)
8. [等等，還有 Chrome 開發人員工具 MCP ！](#)
9. [您可以在 Antigravity 中直接執行這項操作 ！](#)
10. [瀏覽器自動化的其他用途](#)
11. [結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

測試網頁應用程式可能很麻煩。傳統 UI 測試經常會感覺像是一場與脆弱性的持續戰鬥。您發現自己要編寫複雜的指令碼、管理脆弱的 CSS 和 XPath 選取器，以及費盡心思才能驗證簡單的使用者流程。

但如果只要用自然語言告訴代理要測試的內容，代理就會自動執行測試呢？



在本程式碼實驗室中，我們將探索如何使用 Gemini CLI 和 BrowserMCP 等多模態工具。您將瞭解如何使用自然語言建立及執行自動化 UI 測試。請注意，本程式碼研究室不需要事先瞭解 UI 測試架構和工具。

課程內容

- 瞭解 Model Context Protocol (MCP) 的用途，以及這項技術為何能帶來重大變革。
- 瞭解 BrowserMCP 如何讓 AI 代理程式控制網路瀏覽器。
- 如何透過 Gemini CLI 執行自動化 UI 測試。
- 瞭解代理程式技能及其優點。
- 教導代理程式使用 Playwright 技能。
- 同時運用 Google Chrome 開發人員工具 MCP 和技能。
- 快速瞭解 Antigravity 瀏覽器子代理程式。
- 瀏覽器控制項的其他用途。

學習內容

- 設定您的開發環境。
 - 探索需要測試的示範應用程式。
 - 使用 Gemini CLI 透過 BrowserMCP 與應用程式互動。
 - 使用代理程式技能，教導代理程式如何使用 Playwright。
-

步驟 2 · 約 0 分鐘

2. 必要條件

開始瞭解有趣的功能前，請先確認您已備妥所需的一切。

本程式碼實驗室會使用 [Gemini CLI](#)、MCP 工具、代理程式技能和 React 範例應用程式。

工具

本實驗室假設您已具備下列條件：

- Chrome 瀏覽器
- NodeJS
- Gemini CLI
- Git

如要使用 Gemini CLI，請向 [Google 進行驗證](#)。方法有很多種，但我們建議直接使用「使用 Google 帳戶登入」選項。這個選項提供大量免費的 Gemini 使用配額，不需要 Google Cloud 專案。如果您在完成程式碼研究室時使用這個選項，就不會產生任何費用。(如果您已有 Gemini API 金鑰，也可以使用該金鑰)。

這些操作說明假設您是在 Linux (或 WSL) 或 macOS 環境中作業。如果您使用 Windows (像我一樣)，可以透過 [WSL](#) 進行操作。

(請注意，

BrowserMCP 無法透過 Google Cloud Shell 運作

，因為它只會連線至在同一部機器上執行的本機瀏覽器。

設定開發環境

我已在 GitHub 建立示範存放區。其中包含可用於 UI 測試的範例應用程式。請在本機終端機執行下列指令，複製這個存放區：

```
git clone https://github.com/derailed-dash/agent-ic-ui-testing
cd agent-ic-ui-testing
```

我們提供 Makefile，方便您設定環境來啟動試用版應用程式。請執行該檔案來初始化環境：

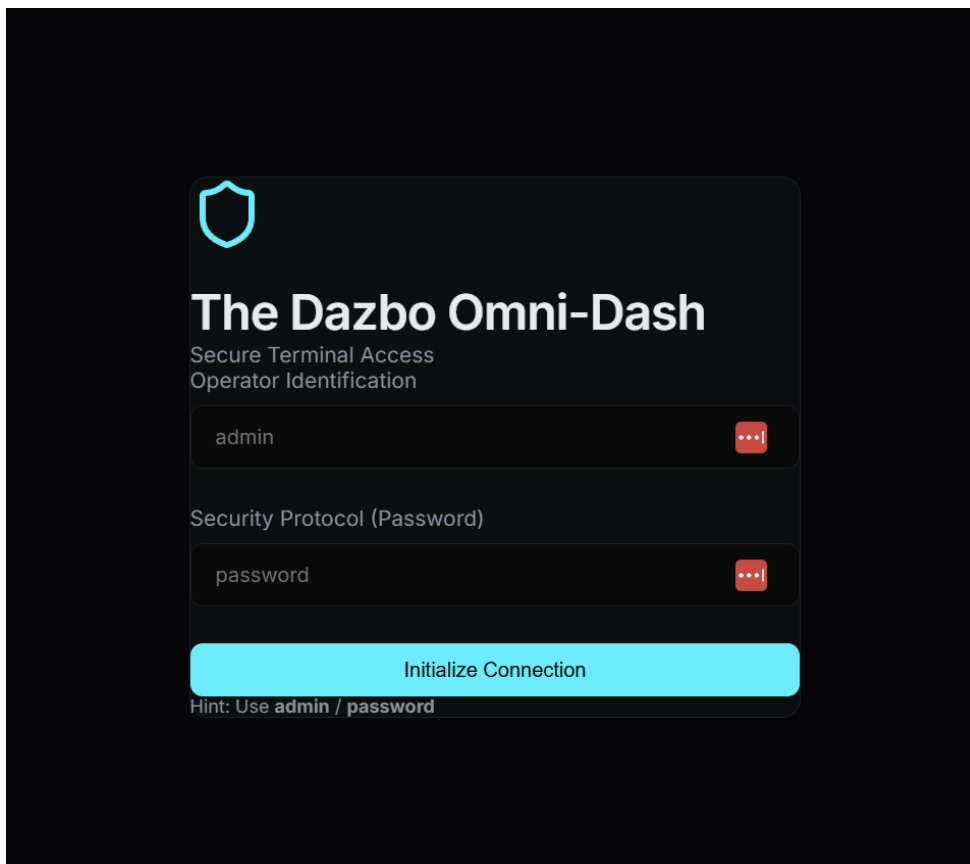
```
make install

# Or if you don't have make
npm install --prefix demo-app
```

步驟 3 · 約 0 分鐘

3. 我們的示範應用程式

我們今天測試的應用程式是「The Dazbo Omni-Dash」，這款未來感十足的深色主題資訊主頁，可管理安全性遙測資料。(沒錯，這是直覺式程式開發！)



為什麼要使用這個應用程式？

這項工具可提供逼真的測試介面，包括：

- 模擬驗證：需要特定憑證的登入流程。
- 動態內容：模擬即時資料的遙測資訊卡和安全性記錄。
- 互動式狀態：導覽選單和表單輸入內容會根據使用者動作而變更。
- 現代化技術：採用 React 和 Vite 建構，提供快速且反應靈敏的體驗。

啟動應用程式

如要啟動應用程式，只要執行下列指令即可：

```
make dev

# Or if you don't have make
npm run dev --prefix demo-app
```

開發伺服器應會很快啟動，應用程式則會位於 <http://localhost:5173>。

```
VITE v7.3.1 ready in 320 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

我們只要點按連結，即可在瀏覽器中開啟應用程式。只要讓這個程序在終端機中執行即可。我們會在另一個終端機工作階段中執行後續的終端機指令。

4. UI 測試的挑戰

傳統 UI 測試向來難以正確執行，維護起來更是困難。常見痛點包括：

- 「不穩定」測試：因時間問題、競爭條件或資產載入緩慢，導致測試結果時而通過、時而失敗。
- 脆弱的選取器：依賴特定 DOM 結構 (例如 `div > div > button`)，只要 UI 稍有調整就會中斷，導致需要不斷維護指令碼。
- 學習曲線陡峭：開發人員必須精通複雜的網域專屬語言和架構專屬的特殊功能 (Cypress、Selenium、Playwright)，才能自動執行基本點擊動作。
- 環境同位：難以重現的應用程式狀態，以及清除測試資料的額外負擔。



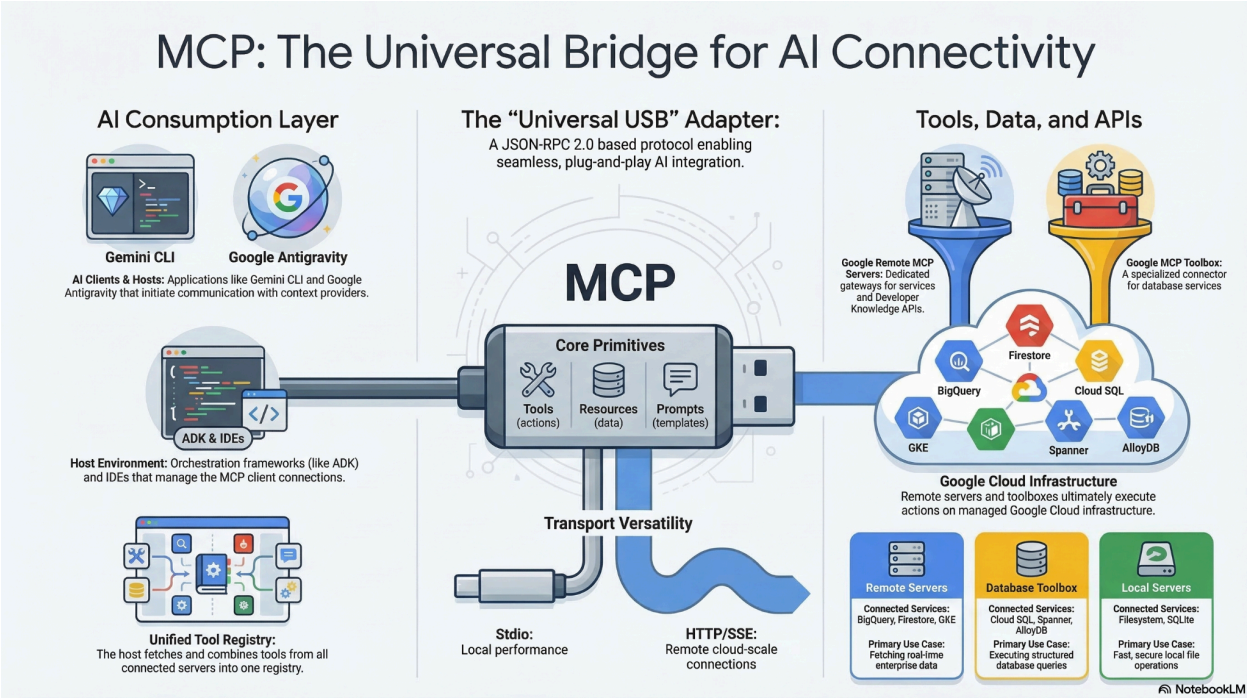
我們需要一種測試方式，著重於「意圖」而非「實作」。

5. MCP 救援

Model Context Protocol (MCP) 是一項開放標準，可讓 AI 模型和代理程式與外部工具、API 和資料互動。這就像通用整合介面，可讓模型和代理尋找並執行有權存取的工具。

傳統上，如要將大型語言模型 (LLM) 與外部資料和工具整合，開發人員必須為每個新資料來源編寫自訂的硬式編碼 API 連線，造成難以持續的「M x N」整合問題，也就是每新增一個模型和工具，維護負擔就會倍增。Model Context Protocol (MCP) 可解決這個問題，因為您不必編寫特定程式碼來協調這些功能。開發人員不必明確編寫複雜的執行工作流程，而是依賴 LLM 解讀使用者的自然語言要求，並動態推論要使用哪些工具。

當使用者發出自然語言指令 (例如「Navigate to localhost:5173, login as 'admin', and click the Submit button」) (前往 localhost:5173，以「admin」身分登入，然後按一下「Submit」按鈕) 時，LLM 會探索可用功能，並生成結構化要求來叫用特定工具。MCP 用戶端會做為翻譯器，將這項要求傳送至指定的 MCP 伺服器，後者會執行動作或擷取資料，並將內容傳回模型。這樣一來，AI 就能自主運作，開發人員不必硬式編碼特定執行路徑。

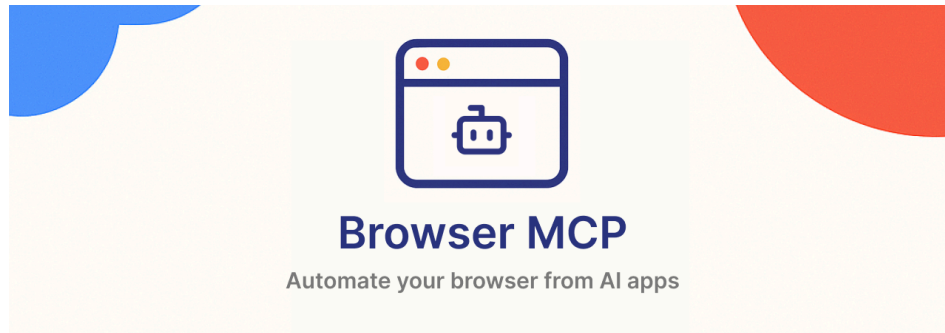


MCP 建立通用標準 (通常稱為「AI 應用程式的 USB-C」)，因此可大幅提升現成可用性。開發人員只要建構一次 MCP 伺服器，任何相容於 MCP 的 AI 主持人都能立即連線，解決 M x N 整合問題。您不必再為每個平台建構自訂 API 橋接器，而是可以運用預先建構的開放原始碼 MCP 伺服器生態系統，用於 GitHub、Slack、資料庫等常見服務，直接將這些伺服器插入代理程式工作流程。這種隨插即用的模組化架構可確保您日後更換 LLM 供應商或升級工具時，核心整合基礎架構完全不會變動。

6. 使用 BrowserMCP 自動化動作

什麼是 BrowserMCP ？

這是我們今天要使用的第一個工具。BrowserMCP 是一種 MCP 伺服器，可為 AI 代理提供與網路瀏覽器互動所需的「眼睛」和「雙手」。簡單來說，這項工具會模擬使用者與瀏覽器的互動。這項工具為開放原始碼，您可以在 [這裡](#) 查看 GitHub 存放區。請參閱[這裡](#)的主要 BrowserMCP 說明文件。



以下列舉部分功能：

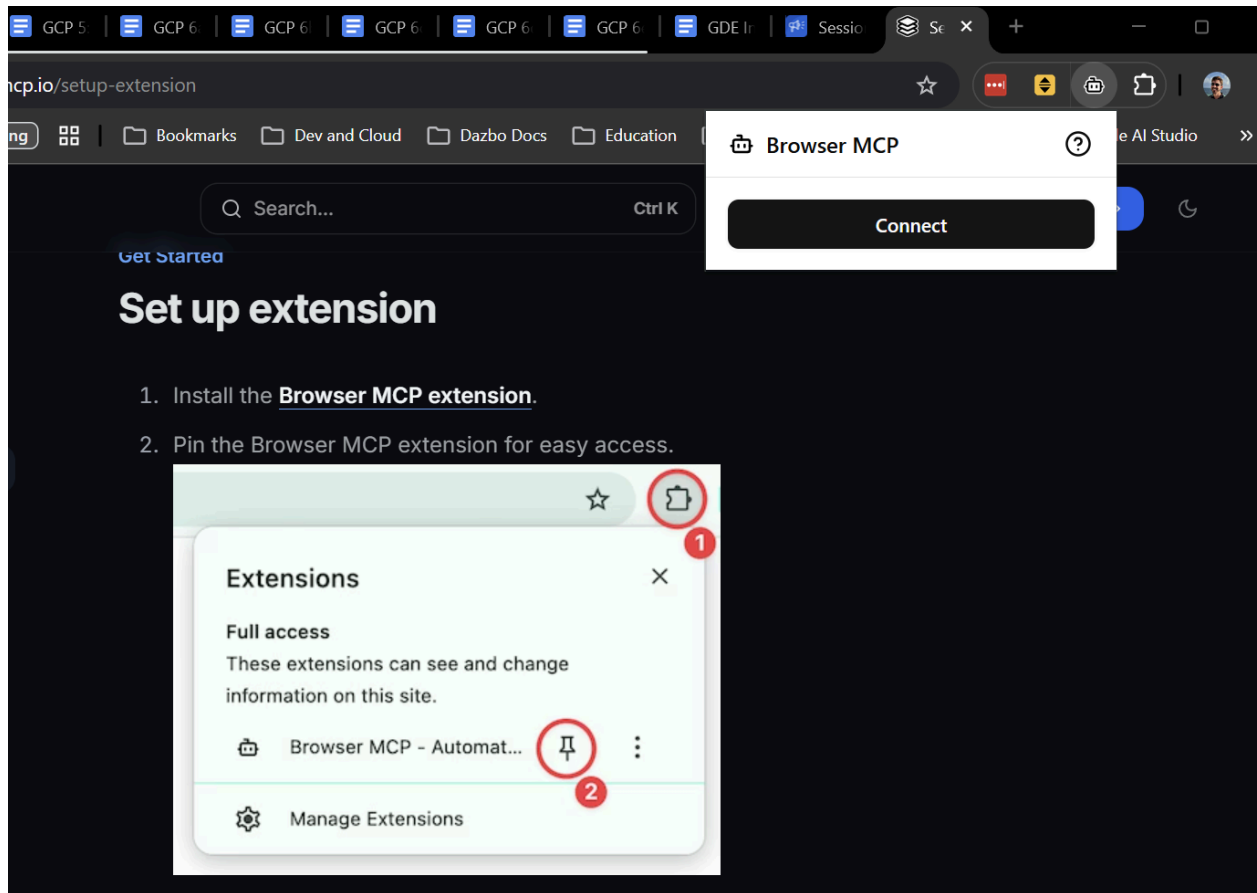
- 可以前往網址。
- 檢查 DOM。
- 可以點選按鈕，並在表單中輸入文字。
- 可以拖曳。
- 可讀取瀏覽器控制台記錄。
- 速度快：自動化作業會在您的電腦上執行。

安裝瀏覽器 MCP

如要使用 BrowserMCP，請完成以下兩個動作：

1. 在 Chrome (或任何以 Chromium 為基礎的瀏覽器) 中安裝 BrowserMCP 擴充功能。
2. 為代理設定 MCP 伺服器。

如要安裝擴充功能，請按照[這裡](#)的指示操作。這項作業只需幾秒鐘就能完成。安裝完畢後，請點選擴充功能的「連線」，允許代理程式控制目前的分頁。(顯然，您希望目前的分頁是執行示範應用程式的分頁！)



接著，我們需要在用戶端中新增實際的 BrowserMCP 伺服器。在 Gemini CLI 中，這項操作非常簡單。只要安裝擴充功能即可：

```
gemini extensions install https://github.com/derailed-dash/browsermcp-ext
```

使用 BrowserMCP 進行測試

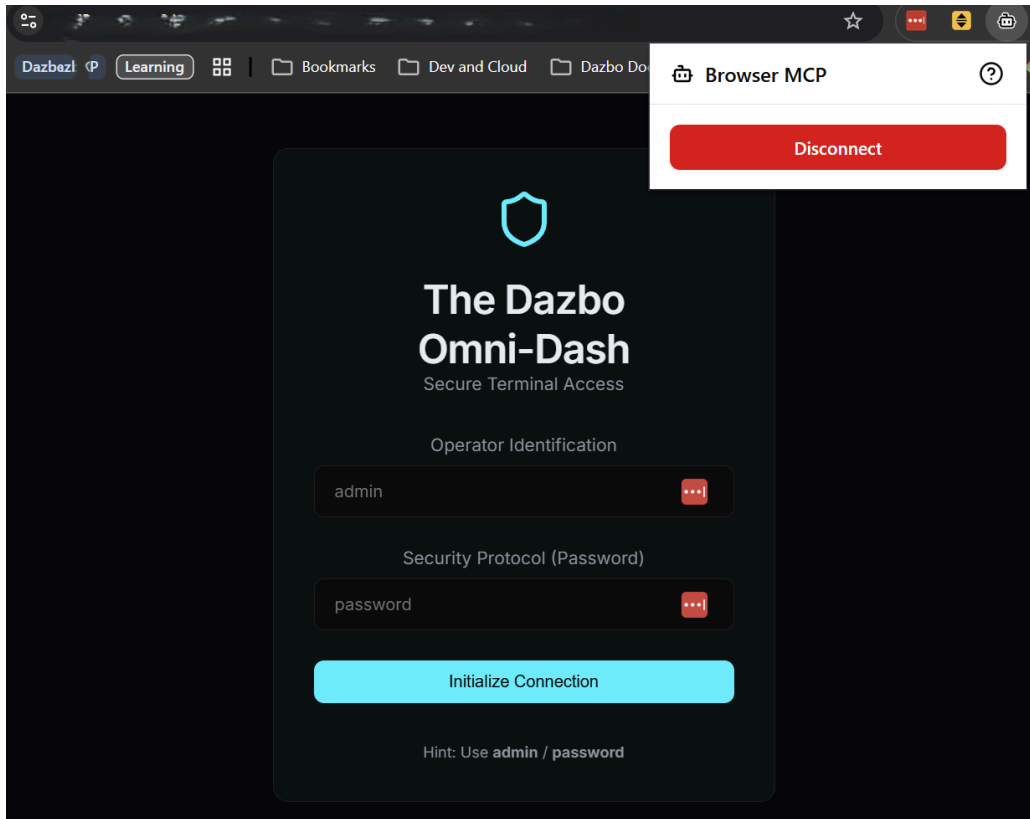
現在開始變魔術。首先，請在新的終端機工作階段中啟動 Gemini CLI (執行 `gemini`)。(請回想一下，示範應用程式是在初始終端機工作階段中執行)。在 Gemini CLI 中執行 `/mcp`，確認安裝作業是否正確。畫面上應會顯示工具清單，如下所示：

```
● browsermcp - Ready (12 tools)
Tools:
- browser_click
- browser_get_console_logs
- browser_go_back
- browser_go_forward
- browser_hover
- browser_navigate
- browser_press_key
- browser_screenshot
- browser_select_option
- browser_snapshot
- browser_type
- browser_wait
```

如果您先前未啟動示範應用程式，請立即啟動：

```
make dev
```

我們需要在 Chrome 瀏覽器中開啟應用程式，並在該分頁中連結 BrowserMCP 擴充功能。按照 `run` 指令中的連結操作。然後按一下 BrowserMCP 擴充功能圖示，再按一下「Connect」(連線)。



現在可以使用 Gemini CLI 執行測試。複製下列提示詞並貼到 Gemini CLI：

```
Using BrowserMCP, connect to the application at http://localhost:5173. If the application is not showing a login screen, first logout. Then login as 'admin' with password 'password', and verify that the dashboard title says 'System Overview'. In the main dashboard, read the telemetry values shown, and present them back to me in a markdown table.
```

Gemini CLI 可能會先檢查範例應用程式是否在指定連接埠上執行。接著，系統會提示您確認工具預計執行的動作：

```
> Using BrowserMCP, connect to the application at http://localhost:5173. If the application is not showing a login screen, first logout. Then login as 'admin' with password 'password', and verify that the dashboard title says 'System Overview'. In the main dashboard, read the telemetry values shown, and present them back to me in a markdown table.

Action Required
? browser_navigate (browsermcp MCP Server) {"url":"http://localhost:5173"}
MCP Server: browsermcp
Tool: browser_navigate
Allow execution of MCP tool "browser_navigate" from server "browsermcp"?

• 1. Allow once
  2. Allow tool for this session
  3. Allow all server tools for this session
  4. No, suggest changes (esc)
```

允許 Gemini CLI 在這個工作階段中執行所有 BrowserMCP 工具。接著返回瀏覽器，觀看自動互動過程！

請注意上述提示詞的幾項重點：

- 如果應用程式已登入，我們會先要求代理程式登出。請注意，我們不需要告訴代理程式點選特定文字，例如「Exit Gateway」。這項功能夠聰明，可判斷要點選的項目。
- 登入並算繪主要頁面後，代理程式會擷取遙測資訊。同樣地，我們不需要告訴代理程式要在特定圖塊中尋找，或比對特定字詞。因此，如果我們稍後擴充或變更這個頁面顯示的資訊，這個提示仍可運作，輸出內容也會擷取到 Markdown 表格中。

很酷吧？

我們暫時不需要 BrowserMCP，請在瀏覽器中**中斷連線**。

步驟 7 · 約 0 分鐘

7. 使用 Skills 和 Playwright 自動化

BrowserMCP 的限制

BrowserMCP 很棒，但有一些限制。例如：

- 您必須先開啟瀏覽器工作階段，並連線至 BrowserMCP 擴充功能，(不會產生新的工作階段)。
- 不支援非 Chromium 瀏覽器。
- 這項功能需要執行獨立的瀏覽器程序，且該程序必須與 MCP 伺服器位於同一部電腦。
- 無法使用本機檔案系統。舉例來說，擴充功能無法建立本機檔案來儲存螢幕截圖，也無法從網頁應用程式下載及儲存檔案 (例如可下載的 PDF 檔案)。
- 這項功能不具確定性。系統會嘗試執行您指示的動作，但本機狀態 (例如非預期的彈出式視窗) 可能會中斷互動。
- 不支援「無頭」作業，也就是說，如果沒有實際的瀏覽器視窗，就無法在 CI/CD 管道中執行。

Playwright

[Playwright](#) 則是更精密的工具。這是一套歷史悠久的開放原始碼瀏覽器自動化和測試架構，它可以執行許多 BrowserMCP 無法執行的動作，包括上述所有項目。

因此更適合執行複雜、可靠且可重複的測試情境。特別適合用於長時間工作階段，或並行執行多個獨立工作階段。

但這類額外功能也代表學習曲線會更加陡峭。

技能

幸好我們不必直接學習如何使用 Playwright。我們可以改用代理程式技能。



那麼，代理程式技能究竟是什麼？這就像是將特定領域的專業知識打包，在 AI 代理需要執行特定工作時提供給它。其中包含操作說明、最佳做法，有時甚至會提供專為特定工作設計的輔助指令碼。

這就是所謂的「漸進式揭露」。與其將所有可想見的 API 文件和測試架構規則塞進 LLM 的初始系統提示 (這會耗盡脈絡窗口，並大量消耗詞元)，代理程式只會在實際需要時讀取技能。這項功能會保持基準背景資訊簡潔明瞭，並即時擷取詳細的「操作說明」。當然，技能絕對可以包含如何運用特定 MCP 伺服器完成工作的指示。

這就像《駭客任務》中的場景：特務看到問題後，意識到需要瞭解 Playwright，於是下載這項技能，然後突然說出「我會功夫」。Boom。即時專家。

若要進一步瞭解技能，請參閱下列文章：

- [教學課程：開始使用 Google Antigravity Skills](#)
- [程式碼研究室：編寫 Google Antigravity Skills](#)

為何 Skills 非常適合 Playwright

在這裡使用技能是絕佳選擇。Playwright 功能非常強大，但語法可能較為複雜。賦予代理程式 Playwright 技能後，我們就不必擔心 LLM 產生過時的語法或編寫脆弱的選取器。我們將提供經過精心設計的權威手冊，詳細說明如何正確使用 Playwright。

我將使用 [Playwright CLI](#) 和相關技能。

我們將在本機安裝 Playwright CLI，然後提供給代理程式使用這項工具所需的知識。為避免任何混淆，我不會安裝任何 Playwright MCP 伺服器。

安裝中

首先，請安裝開放原始碼的 Microsoft Playwright CLI。如果尚未完成，請輸入 `/ quit`` 退出 Gemini CLI。接著在終端機中執行下列操作：

```
# Pre-req: nodejs installed
npm install -g @playwright/cli@latest # Install Playwright CLI globally
npm install @playwright/test # Install Playwright test framework

npx playwright install-deps # Install dependencies
npx playwright install chromium chrome # Install browser binaries in Linux / WSL
```

現在新增技能。這項指令會直接從 GitHub 將技能子資料夾下載到 Gemini 技能資料夾：

```
mkdir -p ~/.gemini/skills
npx degit microsoft/playwright-cli/skills/playwright-cli ~/.gemini/skills/playwright-cli
```

現在可以測試了。

```
# Launch Playwright CLI with visible browser
playwright-cli open https://playwright.dev --headed
```

系統應該會產生瀏覽器工作階段，並開啟指定的網址。

我也希望 Gemini 能夠在「有標題」模式下使用 Playwright，也就是顯示 UI。但這項技能不會告訴 Gemini 如何執行這項操作。因此，我在 **Core** 區段的 `~/.gemini/skills/playwright-cli/SKILL.md` 中新增了以下幾行：

```
# Add the following under the "playwright-cli open" command

# Run in headed mode so we can see the browser
playwright-cli open https://playwright.dev --headed
```

使用 Playwright 進行測試

和先前一樣，我們需要啟動應用程式 (如果尚未執行)。在初始終端機工作階段中執行這項操作：


```
make dev
```

接著，在另一個終端機工作階段中，暫時停用 BrowserMCP，以免代理程式混淆要使用的工具。重新啟動 Gemini CLI，然後執行：

```
/mcp disable browsermcp
```

現在請 Gemini 使用 Playwright 導覽至應用程式。但與 BrowserMCP 不同，我們不需要先啟動瀏覽器。Playwright 會透過本機程序為我們執行這項操作。

在 Gemini CLI 中輸入下列提示詞：

```
Using Playwright, connect to the application at http://localhost:5173. Then login as 'admin' with password 'password', and verify that the dashboard title says 'System Overview'. Take a screenshot of the dashboard and save it to output/dashboard.png. In the main dashboard, read the telemetry values shown, and present them back to me in a markdown table.
```

(與往常一樣，Gemini CLI 會先要求授權，再執行任何工具)。

這兩者有何不同？

- 我們不需要先啟動瀏覽器。
- 我們不需要啟動及連線瀏覽器擴充功能。
- 我們不需要先告知代理程式登出。測試會從「乾淨」的階段例項化。
- 我們可以擷取螢幕截圖，並儲存為本機檔案。

不久後，`output` 資料夾中就會顯示 `dashboard.png` 檔案。

請注意，您會在 Gemini CLI 中看到工具呼叫執行作業，但不會看到瀏覽器 UI。這是因為 Playwright 預設會以「無頭模式」執行。

但如果使用這個修正後的提示重新執行，您也能看到 UI：

```
Using Playwright, connect to the application at http://localhost:5173 in **headed** mode, and keep the browser open when you're done. Login as 'admin' with password 'password', and verify that the dashboard title says 'System Overview'. Take a screenshot of the dashboard and save it to output/dashboard.png. In the main dashboard, read the telemetry values shown and record them. Then wait 3 seconds, read them again. Now present the data back to me in a markdown table.
```

不久後，Gemini CLI 輸出內容應如下所示：

✦ I have completed the task in headed mode. The browser remains open, the dashboard title 'System Overview' has been verified, and the screenshot is saved at `output/dashboard.png`.

The telemetry values from the dashboard are:

Metric	Value	Status
Active Threats	0	Perimeter Secure
System Uptime	99.9%	Peak Efficiency
Network Throughput	1.2Gbps	Optimal

這是不是很棒？

步驟 8 · 約 0 分鐘

8. 等等，還有 Chrome 開發人員工具 MCP ！

Chrome 開發人員工具是內建於 Chrome 瀏覽器的一組網頁程式開發人員工具，適用於網頁開發和偵錯。這項技術已存在許久，也就是在 Chrome 中開啟「更多工具」->「開發人員工具」時，可以互動的控制台。

但現在 Gemini CLI 有自己的 [MCP 伺服器](#)，去年考慮透過 Gemini CLI 進行瀏覽器自動化時，這項功能還不存在。但現在，您可以使用 BrowserMCP 執行所有操作，以及使用 Playwright 執行大部分操作，不必在瀏覽器中安裝任何項目，也不必安裝本機 CLI。

立即試試吧！

目前我們已驗證過，確認可在 Google Cloud Shell 中運作。因此，我們將使用 [Google Cloud 控制台](#) 中的 Google Cloud Shell。

開啟控制台並啟動 Cloud Shell 工作階段。接著：

```
# Clone the sample app - like we did before
git clone https://github.com/derailed-dash/agentik-ui-testing
cd agentik-ui-testing

# Build the application - like we did before
make install

# Install the Chrome DevTools MCP server Gemini CLI Extension
gemini extensions install https://github.com/ChromeDevTools/chrome-devtools-mcp
```

現在我們需要在 Cloud Shell 中安裝 Chrome 可執行檔：

```
# Get the latest executable for Ubuntu
wget https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb

# Install it
sudo apt install ./google-chrome-stable_current_amd64.deb -y

# Check it and get the executable path
which google-chrome

# Cleanup
rm google-chrome-stable_current_amd64.deb
```

最後一個步驟：我們需要告知 Chrome 開發人員工具 MCP 伺服器 Chrome 可執行檔的位置。只要在 MCP 伺服器設定中設定 `executable-path` 選項，並將其設為 `headless`，即可完成這項操作。如要完成這項操作，請編輯 `~/.gemini/extensions/chrome-devtools-mcp/gemini-extension.json` 檔案：

```
{
  "name": "chrome-devtools-mcp",
  "version": "latest",
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": [
        "-y",
        "chrome-devtools-mcp@latest",
        "--executable-path=/usr/bin/google-chrome",
        "--headless"
      ]
    }
  }
}
```

太好了！一切就緒後，請從 Cloud Shell 啟動 `gemini`，然後使用 `/mcp list` 指令檢查 MCP 伺服器是否正在執行 (如先前所述)。

最後，我們準備好使用提示詞進行測試。

我們來試試其他做法。這次，我們要請 Gemini CLI 啟動並連線至示範應用程式：

```
Launch my demo application with `make dev`. Then, using Chrome DevTools MCP, connect to the application at the exposed localhost URL. Login as 'admin' with password 'password', and verify that the dashboard title says 'System Overview'. Take a screenshot of the dashboard and save it to output/dashboard.png. In the main dashboard, read the telemetry values shown, and present them back to me in a markdown table.
```

系統會照常提示您允許執行 MCP 伺服器。但您也會發現，這項要求會嘗試啟動技能。沒錯，這個擴充功能包含 MCP 伺服器，以及引導代理如何善用 MCP 伺服器的技能。太棒了！

幾秒後，Gemini CLI 應會在表格中顯示結果，並儲存螢幕截圖。您可以從 Cloud Shell 下載螢幕截圖，確認畫面是否正常。

```
✚ I will read the dashboard screenshot from the temporary directory.

✓ ReadFile ../../dashboard_tmp.png
Read image file: ../../gemini/tmp/agentui-testing-1/dashboard_tmp.png

✚ Here is the dashboard screenshot. I've also updated the telemetry table with the latest values:
```

Metric	Value	Note
Status	OPTIMAL	-
ACTIVE THREATS	0	Perimeter Secure
SYSTEM UPTIME	99.9%	Peak Efficiency
NETWORK THROUGHPUT	1.15 Gbps	-

```
The screenshot has been saved to output/dashboard.png.

? for shortcuts

Shift+Tab to accept edits    2 GEMINI.md files | 2 MCP servers | 7 skills | 1 Background process

> | Type your message or @path/to/file

workspace (/directory)      branch
~/agentui-testing           main

Transferred 1 item

dashboard.png                /home/derailed_dash/agentui-testing/...
```

9. 您可以在 Antigravity 中直接執行這項操作！

Google Antigravity 包含 [Browser Subagent](#)，可提供類似 Playwright CLI 的功能。在 Antigravity 中問問 Gemini 以互動方式啟動網址時，系統會自動啟動這個子代理程式。

這個子代理程式會接收您的概略目標 (例如「檢查登入表單是否正常運作」)，透過螢幕截圖和 DOM 視覺化分析頁面版面配置，並自行判斷點擊和按鍵操作。這項技術基本上就是視覺化多模態 AI，可像人類一樣瀏覽網路。最棒的是，代理會自動錄製影片和擷取螢幕截圖，記錄自己執行的所有動作，並直接儲存到本機工作區，做為完成工作的視覺證據。Antigravity 將這類視覺證據稱為「Artifacts」。

WSL 使用者注意事項：在 Antigravity 中讓瀏覽器代理程式運作有點麻煩。我已 [讓它運作](#)，但發現子代理程式在這個環境中不穩定且不可靠。這也是我愛用 Playwright CLI 的原因之一！

10. 瀏覽器自動化的其他用途

瀏覽器自動化不僅能確保登入按鈕在週五下午部署前正常運作，一旦發現可以直接將 LLM 連線至瀏覽器，您就能自行開發各種代理程式專案。

如果您要自行建構 AI 代理程式，可以運用 BrowserMCP 或 Playwright CLI 等工具，執行下列繁重工作：

- 個人研究助理：假設您要求代理程式研究特定網址的主題，但該網站需要登入，且選單複雜難以操作。您不必編寫下週會失效的自訂網頁擷取工具，只要告訴代理登入、前往資料，然後為您摘要資料即可。
- 「旋轉椅」整合人員：我們都有沒有 API 的舊版內部網路系統。您一定知道，就是必須手動從系統 A 複製資料，然後貼到系統 B 的表單中。具備瀏覽器自動化功能的代理程式可做為通用黏合劑，讀取舊版系統的畫面，並在新系統中填寫表單。
- 自動分類和補救：凌晨 3 點收到監控系統的 P1 警示？您的代理程式可以自動開啟特定資訊主頁網址、讀取圖表或記錄 (使用多模態視覺功能)，並直接在 Slack 頻道中發布摘要，在事件發生期間為您節省寶貴的時間。

這種做法的優點在於，您不再受限於可用的 API。如果使用者可以在瀏覽器中完成某項操作，您的代理程式也能做到。

11. 結論

恭喜！您只要以簡單的英文向 AI 代理說明想執行的動作，就能建立並執行自動化且穩健的 UI 測試。沒有容易出錯的 CSS 選取器，也沒有複雜的設定指令碼。

您已學會：

- UI 測試不一定很麻煩：只要專注於測試意圖，而非脆弱的 DOM 實作，就能大幅減少維護負擔。
- Model Context Protocol (MCP) 可讓代理程式即插即用，存取各種工具、資料和環境。
- BrowserMCP 是一項強大的工具，可將代理能力帶入您現有的本機 Chrome 工作階段。
- 透過 Skills 和 Playwright CLI，您可運用漸進式揭露功能，進行可重複的確定性自動化測試，達到全新境界。
- Antigravity 的瀏覽器子代理程式更進一步，直接提供自主式多模態導覽和構件記錄功能。

現在就開始自動執行無聊的工作吧！

Useful Links

如要深入瞭解今天介紹的工具和概念，請參閱下列資源：

存放區代碼

- [agentic-ui-testing GitHub 存放區](#) - 如果您覺得本程式碼研究室很有幫助，請為存放區加上星號！

核心工具與架構

- [BrowserMCP GitHub 存放區](#)
- [BrowserMCP 說明文件](#)
- [BrowserMCP Gemini CLI 擴充功能](#) - 如果您覺得這個程式碼研究室很有用，請為存放區加上星號！
- [Playwright](#)
- [Google AI Studio](#)
- [Chrome 開發人員工具](#)
- [Chrome 開發人員工具 MCP](#)

代理概念與技能

- [教學課程：開始使用 Google Antigravity Skills](#)
- [程式碼研究室：開始使用 Antigravity 技能](#)
- [Dazbo 的原創網誌：在幾秒內建立自動化 UI 測試](#)

其他

- [在 WSL 中使用 Google Antigravity](#)
 - [Dazbo 在 Medium 上的文章](#)
 - [Dazbo 的 LinkedIn 頁面](#)
-